

PLAN DE IMPLEMENTACIÓN ENTORNO DE DESARROLLO MÓVIL

PROYECTO: TAG-OK

Tabla de contenidos

Historial de revisiones	3
Introducción	3
Resumen Ejecutivo	3
Alcance de la implementación	4
Elementos del entorno	4
Herramientas y Tecnologías involucradas	5
Preparación del ambiente de desarrollo	6
Configuración del proyecto	6
Configuración de claves y variables de entorno	6
Verificación de implementación	8
Resolución de problemas comunes	8

Historial de revisiones

Fecha	Versión	Descripción	Autor
20 de Junio de 2026	1.0	Versión inicial del documento	Diego Rodríguez, Paulina Troncoso, Ricardo Sánchez

Introducción

El presente documento describe el procedimiento completo para la implementación y configuración del entorno de desarrollo del módulo móvil del proyecto TAG OK. El objetivo es establecer una guía estandarizada que permita a cualquier miembro del equipo preparar su estación de trabajo de manera rápida y consistente, eliminando problemas de configuración y asegurando la compatibilidad en todo el ciclo de vida del desarrollo.

Este plan cubre desde la instalación de las herramientas base hasta la configuración específica del proyecto, incluyendo la gestión de claves de API y la conexión con el backend. Está dirigido al equipo de desarrollo y a los evaluadores del proyecto académico, garantizando la reproducibilidad del entorno de trabajo.

Resumen Ejecutivo

Para desarrollar la aplicación móvil de TAG OK, construida en Kotlin con Jetpack Compose, se requiere un entorno de desarrollo estandarizado. Este plan describe la implementación de dicho entorno en tres fases principales.

La primera fase consiste en la preparación del entorno base, que incluye la instalación de Android Studio, el JDK 21 y el SDK de Android con nivel de API 34. La segunda fase abarca la configuración del proyecto mediante la clonación del repositorio y la sincronización inicial con Gradle para descargar todas las dependencias necesarias. La tercera fase se enfoca en la gestión de secretos, configurando las variables de entorno y archivos de propiedades locales para una conexión segura con los servicios externos, sin exponer credenciales en el repositorio.

El resultado es una guía paso a paso que, al ser seguida, permite tener una instancia del proyecto tag-ok-app ejecutándose en un emulador o dispositivo físico y conectada correctamente al ecosistema de servicios del proyecto.

Alcance de la implementación

Este documento abarca la configuración del entorno necesario para compilar y ejecutar la aplicación tag-ok-app. El alcance incluye la instalación del IDE de desarrollo Android Studio en su versión más reciente compatible con el proyecto, la configuración del Kit de Desarrollo de Software de Android, la clonación y preparación inicial del repositorio de código fuente, la configuración de las dependencias del proyecto mediante Gradle, el establecimiento de las claves y tokens necesarios para la comunicación con servicios de terceros, y la validación de la configuración mediante la compilación y ejecución exitosa en un emulador.

Queda fuera del alcance la configuración del entorno de desarrollo del backend o del frontend web, así como la configuración de la infraestructura en Supabase y la base de datos PostgreSQL con PostGIS.

Elementos del entorno

El entorno de desarrollo está compuesto por los siguientes elementos de software y configuraciones. Como sistema operativo base se utiliza Windows 10/11, macOS o Linux. El IDE principal es Android Studio, acompañado del JDK 21. El SDK de Android incluye la plataforma Android 14.0 con nivel de API 34 y las Build-Tools en su versión 34.0.0 o superior. Para las pruebas se utiliza un Dispositivo Virtual Android con imagen del sistema API 34.

Las herramientas de build incluyen Gradle 8.12.0 administrado mediante el Gradle Wrapper y el plugin de Android para Gradle en su versión 8.12.0. El control de versiones se realiza

con Git. Los servicios externos utilizados son Supabase para autenticación y base de datos, y Mapbox para la visualización de mapas.

Herramientas y Tecnologías involucradas

Las herramientas utilizadas para la implementación del entorno son Android Studio como IDE oficial para el desarrollo en Android, que provee el editor de código, el administrador de SDK, el gestor de emuladores y las herramientas de depuración. El Gradle Wrapper permite ejecutar tareas de Gradle sin una instalación global, asegurando que todos los desarrolladores usen la misma versión. El Administrador de SDK es la herramienta interna de Android Studio para descargar las plataformas, build-tools y herramientas de línea de comandos necesarias.

El proyecto utiliza un catálogo de versiones centralizado mediante el archivo `libs.versions.toml`, lo que permite gestionar todas las dependencias desde un único punto. Las variables de entorno y tokens se gestionan mediante el archivo `local.properties`, el cual está excluido del control de versiones para proteger las credenciales del proyecto.

Preparación del Ambiente de Desarrollo

El primer paso consiste en instalar Android Studio descargando el instalador desde el sitio oficial de Android Developers. Durante la instalación, se debe aceptar la configuración de tipo Standard en el asistente de inicio, lo que descargará automáticamente el JDK 21 y los componentes más recientes del SDK de Android.

Una vez instalado, se debe verificar y configurar el SDK de Android accediendo a la sección de configuración del IDE. En la pestaña de SDK Platforms se debe marcar la casilla para mostrar los detalles del paquete y asegurar que para Android 14.0 esté instalado el Android SDK Platform 34. En la pestaña de SDK Tools se debe verificar la instalación de Android SDK Build-Tools versión 34 o superior, Android Emulator, Android SDK Platform-Tools y el acelerador de emulador correspondiente al procesador.

El tercer paso es crear un dispositivo virtual mediante el Administrador de Dispositivos. Se recomienda seleccionar un hardware como Pixel 7, utilizar una imagen del sistema para API 34 y asignar un nombre descriptivo al dispositivo virtual. Una vez creado, se debe iniciar para verificar su correcto funcionamiento.

Preparación del ambiente de desarrollo

El primer paso consiste en instalar Android Studio descargando el instalador desde el sitio oficial de Android Developers. Durante la instalación, se debe aceptar la configuración de tipo Standard en el asistente de inicio, lo que descargará automáticamente el JDK 21 y los componentes más recientes del SDK de Android.

Una vez instalado, se debe verificar y configurar el SDK de Android accediendo a la sección de configuración del IDE. En la pestaña de SDK Platforms se debe marcar la casilla para mostrar los detalles del paquete y asegurar que para Android 14.0 esté instalado el Android SDK Platform 34. En la pestaña de SDK Tools se debe verificar la instalación de Android SDK Build-Tools versión 34 o superior, Android Emulator, Android SDK Platform-Tools y el acelerador de emulador correspondiente al procesador.

El tercer paso es crear un dispositivo virtual mediante el Administrador de Dispositivos. Se recomienda seleccionar un hardware como Pixel 7, utilizar una imagen del sistema para API 34 y asignar un nombre descriptivo al dispositivo virtual. Una vez creado, se debe iniciar para verificar su correcto funcionamiento.

Configuración del proyecto

Con el entorno base listo, se procede a clonar el repositorio. Desde una terminal, se debe ejecutar el comando de clonación apuntando a la URL del repositorio del proyecto. Una vez clonado, se navega hasta la carpeta tag-ok-app que contiene el módulo móvil.

A continuación, se abre Android Studio y se selecciona la opción para abrir un proyecto existente, navegando hasta la carpeta tag-ok-app. El IDE comenzará a indexar el proyecto y solicitará una sincronización de Gradle. Se debe esperar a que este proceso termine exitosamente, lo cual se confirma con un indicador verde en la barra inferior de Build.

Configuración de claves y variables de entorno

Este paso es crítico para la seguridad del proyecto. Las claves nunca deben incluirse en el repositorio Git. Se debe verificar que el archivo local.properties esté presente en la raíz del proyecto tag-ok-app. Este archivo es ignorado por Git de forma predeterminada.

En el archivo `local.properties` se debe verificar la ruta al SDK de Android y añadir las claves necesarias para los servicios externos. La estructura del archivo debe ser la siguiente:

La primera línea es un comentario que indica que este archivo no debe ser incluido en sistemas de control de versiones, ya que contiene información específica de la configuración local. La segunda línea indica la ubicación del SDK, utilizada únicamente por Gradle. La tercera línea es otro comentario sobre personalización al usar un sistema de control de versiones. Luego se incluye la fecha de modificación del archivo.

A continuación se definen las variables de entorno. `MAPBOX_ACCESS_TOKEN` contiene el token público de acceso a los servicios de Mapbox, necesario para la renderización de mapas y el cálculo de rutas en la aplicación. `SDK_REGISTRY_TOKEN` contiene el token de registro del SDK de Mapbox, requerido para la descarga de las dependencias de Mapbox desde su repositorio privado durante el proceso de build.

Para que la aplicación pueda utilizar el token de Mapbox, se configura en el archivo `build.gradle.kts` del módulo `app`. Se añade un bloque de código que lee las propiedades del archivo `local.properties` y las expone como recursos de string. Específicamente, el token de Mapbox se configura como un recurso de string para ser referenciado desde el `AndroidManifest.xml`.

Adicionalmente, el token de registro del SDK se utiliza en la configuración de repositorios de Gradle para autenticar la descarga de las dependencias de Mapbox. Esta configuración se realiza en el archivo `settings.gradle.kts` del proyecto, donde se declara el repositorio de Mapbox con las credenciales correspondientes.

Una vez añadidas estas configuraciones, se debe realizar una nueva sincronización del proyecto con Gradle para que los cambios surtan efecto.

Regalo =)

```
MAPBOX_ACCESS_TOKEN=pk.eyJ1IjoidGFnLW9rLW12cDIwMjYiLCJhIjoiY21vOTAxag1MDQzaTJzcTRhNTd0Z3hzbSJ9.O6tYvpTSiVG6OLJvV7Anng
SDK_REGISTRY_TOKEN=sk.eyJ1IjoidGFnLW9rLW12cDIwMjYiLCJhIjoiY21vajFvd3U3MDAxMDJwcHNuOGNwOWxzczSJ9.HbAftx-ZQgiWAWLEwFBM9g
```

Verificación de implementación

Para confirmar que todo está configurado correctamente, se debe realizar una limpieza del proyecto seguida de una reconstrucción completa desde el menú Build de Android Studio. El proceso de build debe ser exitoso y no mostrar errores en la consola.

Posteriormente, se debe ejecutar la aplicación en el emulador previamente configurado. Se verifica que la aplicación se instale correctamente, que la pantalla de inicio de sesión se muestre sin errores y que la funcionalidad de autenticación con Supabase responda adecuadamente. También se comprueba que el mapa de Mapbox se renderice correctamente, confirmando que el token fue configurado de forma adecuada.

Adicionalmente, se debe verificar que la conexión con el backend sea exitosa. Si el backend se está ejecutando localmente en el puerto 8000, es necesario establecer la conexión mediante el comando `adb reverse tcp:8000 tcp:8000` para dispositivos físicos, o configurar la URL base en la aplicación para emuladores.

Resolución de problemas comunes

Durante el proceso de configuración pueden presentarse diversos errores. Si el comando `adb` no se reconoce en la terminal, significa que la herramienta de línea de comandos de Android no está en las variables de entorno del sistema. Para solucionarlo se debe usar la ruta completa al ejecutable `adb` que se encuentra dentro de la carpeta `platform-tools` del SDK de Android, o bien agregar esta ruta a la variable `Path` del sistema.

Si la aplicación en el emulador no logra conectarse al backend, el problema suele ser que el emulador utiliza la dirección especial `10.0.2.2` para referirse al equipo host en lugar de `localhost`. En este caso, se debe modificar temporalmente la constante `BASE_URL` en el archivo `RouteApiService.kt` para que apunte a dicha dirección IP, recordando revertir el cambio antes de hacer commit al repositorio.

Si la sincronización de Gradle falla con errores de dependencias de Mapbox, se debe verificar que el `SDK_REGISTRY_TOKEN` esté correctamente configurado en el archivo `local.properties`. Sin este token, Gradle no puede autenticarse contra el repositorio privado de Mapbox y la descarga de las dependencias fallará.

Si el mapa no se muestra en la aplicación, se debe verificar que el `MAPBOX_ACCESS_TOKEN` esté correctamente configurado en el archivo `local.properties`.

y que el recurso de string correspondiente esté referenciado en el `AndroidManifest.xml` mediante la etiqueta meta-data con el nombre `com.mapbox.maps.MapboxAccessToken`.

Si la sincronización de Gradle falla con errores de versión del plugin de Android, se debe verificar que la versión declarada en el archivo `libs.versions.toml` sea compatible con la versión de Android Studio instalada. Para este proyecto se utiliza la versión 8.12.0 del plugin de Android para Gradle, la cual es compatible con Android Studio en sus versiones más recientes.